

Benchmark Report

Overview

This project benchmarks the performance of multiple JSON processing solutions under different workloads and dataset sizes. The primary goal is to compare execution speed across different implementations.

The benchmark focuses on common JSON processing operations such as filtering, mapping, and length calculation on generated datasets of varying sizes and nesting depths.

Note: Benchmark results are **not** representative at this time.

Compared Software

Software	Description
Custom Java (Single-threaded)	A custom JSON processor implemented in Java using a single-threaded execution model
Custom Java (Multi-threaded)	A multi-threaded variant of the Java implementation
Custom Python	A Python implementation generated by GitHub Copilot based on my Java implementation. ▼ [prompt] Create a similar json processor in Python to what I created in <code>json_bench_java</code>, with the same commands, but only single-threaded, not multiple engines. The project has a base in <code>json_bench_python</code>. Make it high quality and well structured.
jq	The widely used command-line JSON processor

Motivation Behind Software Selection

The selected software solutions represent different approaches to JSON processing:

- The custom Java implementations allow direct comparison between single-threaded and multi-threaded processing strategies.
- The Python implementation provides insight into the performance characteristics of an interpreted language compared to Java.
- jq serves as an established and highly optimized baseline commonly used for command-line JSON processing tasks.

Software

Operating System	Windows 11 Home
OS Build	25H2
Java Version	21.0.2
Python Version	3.12.0

jq Version	1.8.1
------------	-------

Hardware

CPU	AMD R7 5825U (8C16T @ 2GHz; Boost 4.5GHz)
CPU Cache	L1: 512 KB; L2: 4 MB; L3: 16 MB
Memory	24 GB DDR4 3200 MT/s
Storage	Samsung NVMe SSD

Benchmark Environment

The selected software solutions represent different approaches to JSON processing:

- The Java implementations are benchmarked using `JMH`
- The Python implementation is benchmarked using `pyp perf`
- jq is benchmarked using `Hyperfine`

Dataset Generation

Synthetic JSON datasets are generated programmatically using a custom Python generator script.

The datasets vary in:

- File size
- Record count
- Nesting depth
- Structural complexity

Dataset Sizes

Dataset	Record Count	Approximate Size @ no nesting
Small	10k records	0.8 MB
Medium	100k records	8.5 MB
Large	1M records	87 MB
Extra Large	10M records	890 MB

Test Cases

The following operations are benchmarked:

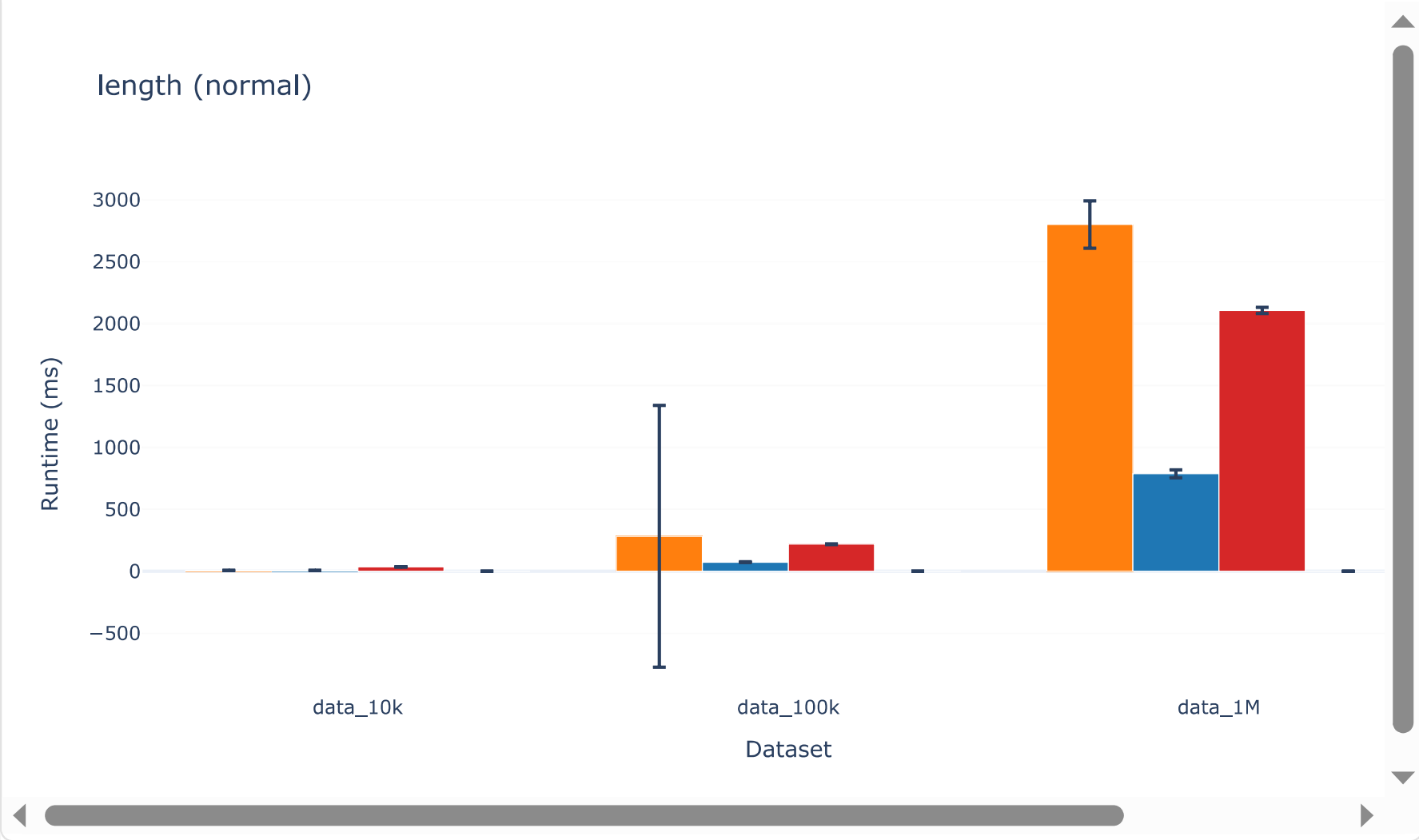
Operation	Description
length	simply counts the number of records

Operation	Description
filter	<code>id < 100 id == 5555 age != 4099 age > 3012</code>
map	applies the following to every value: <code>+100 4 6</code>

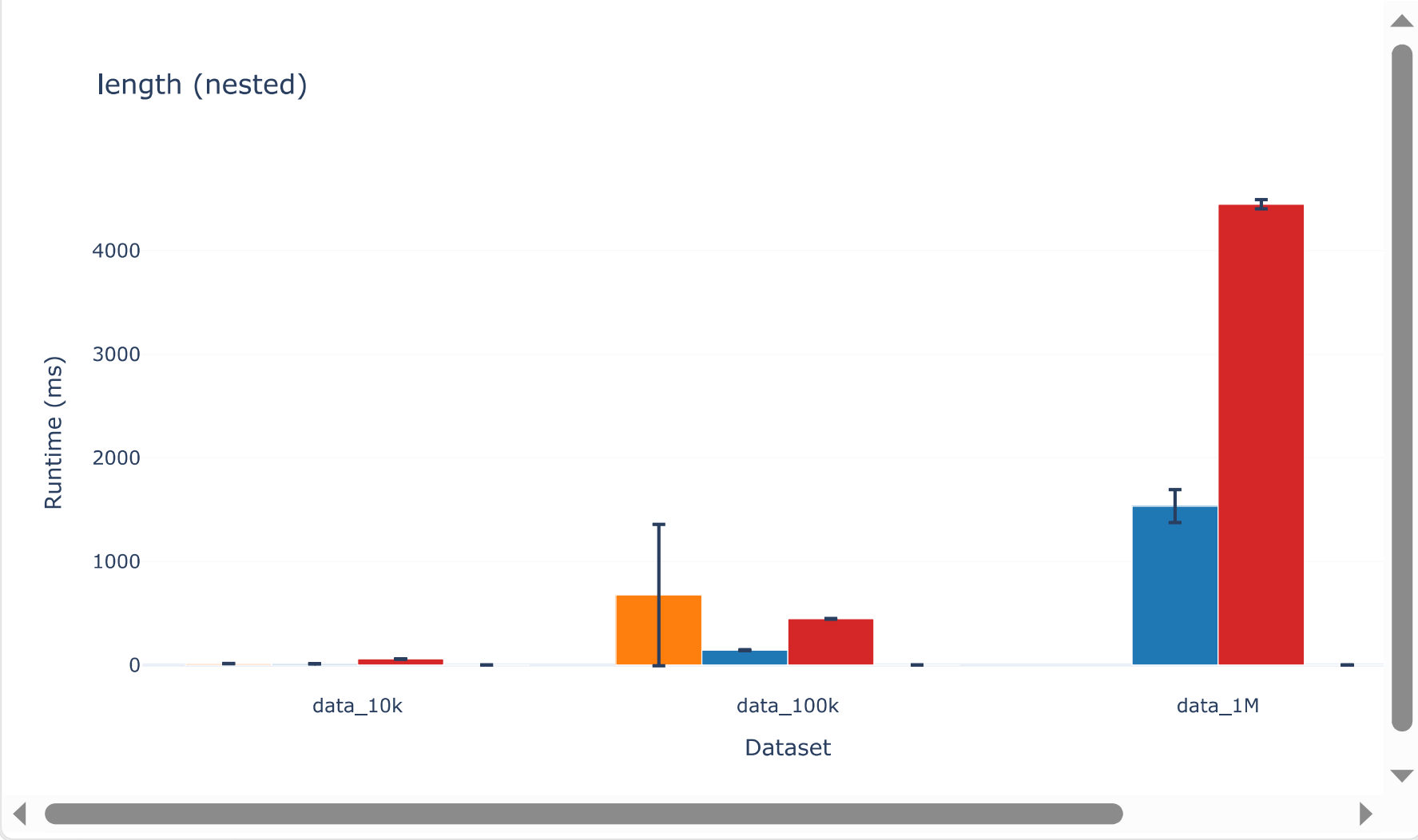
Benchmark results

▼ length

▼ normal



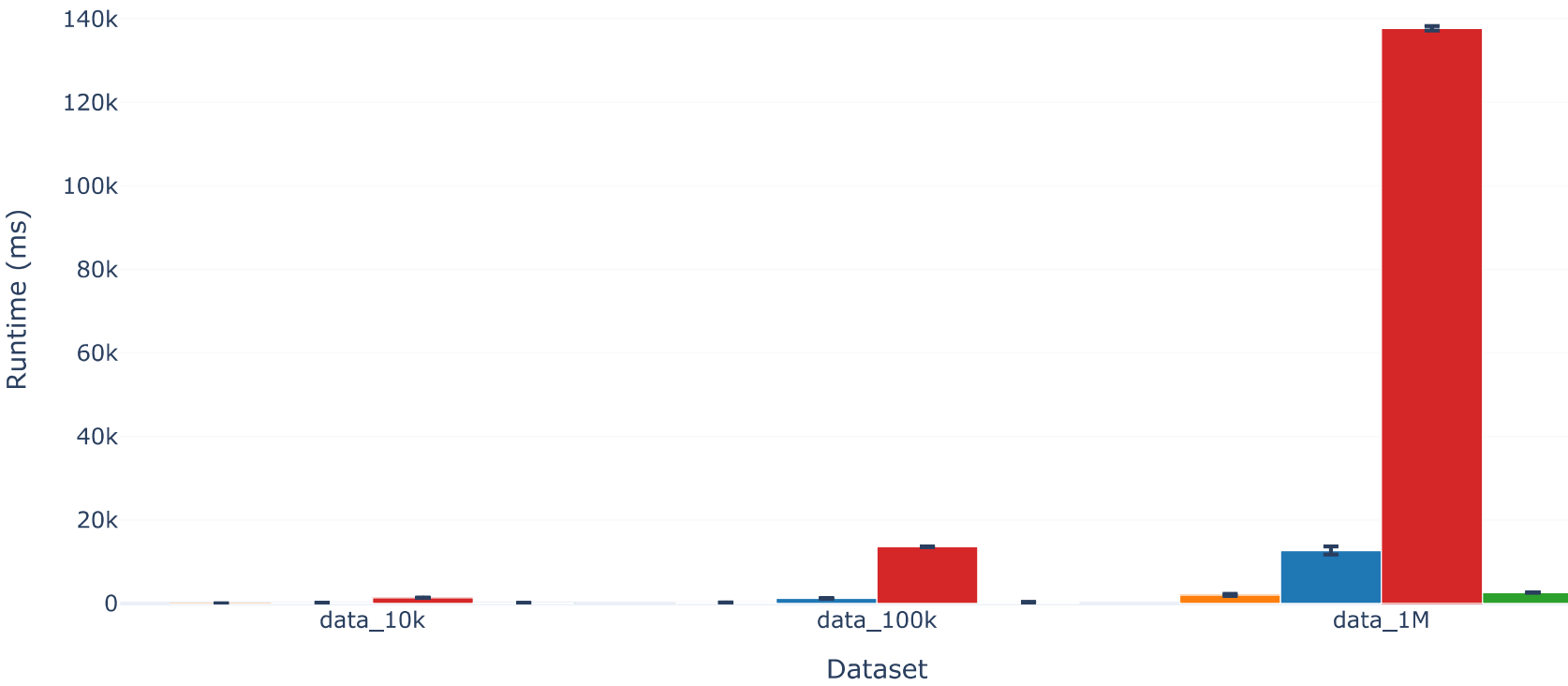
▼ nested



▼ filter_age

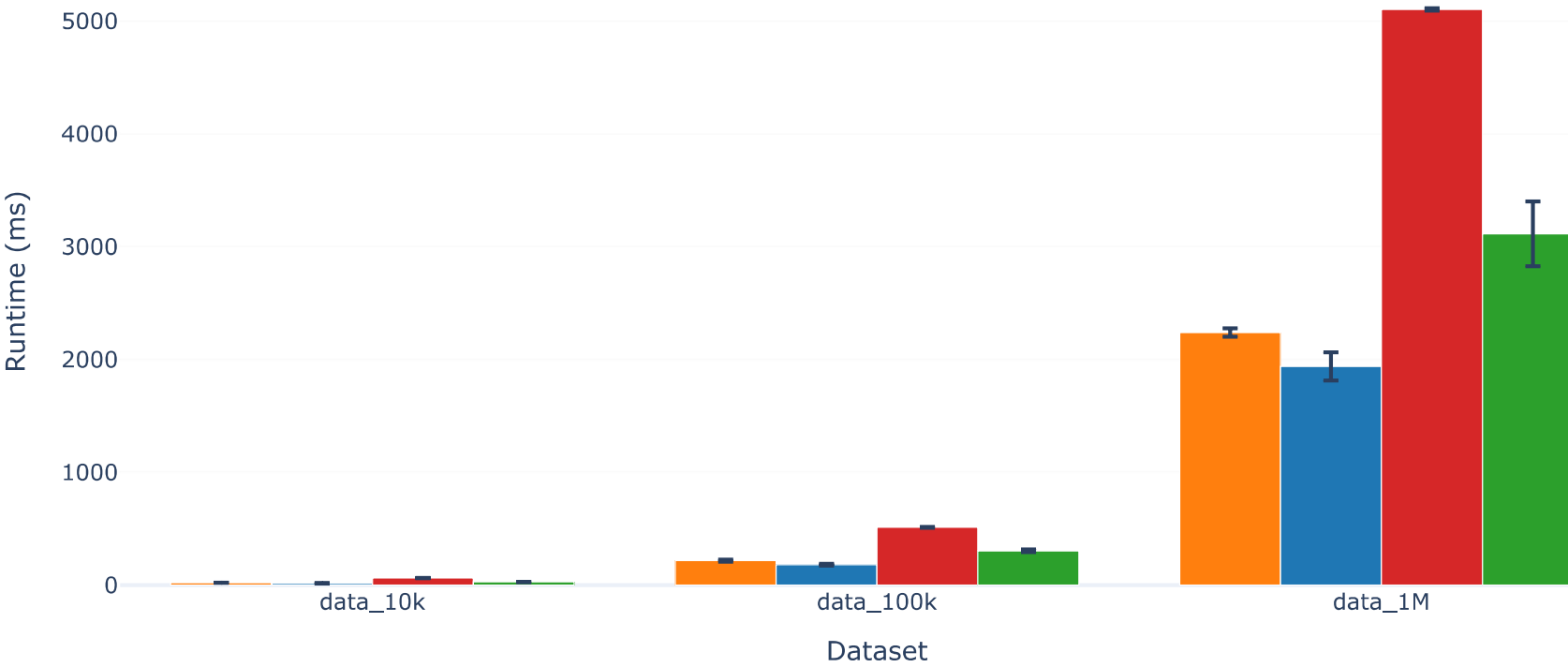
▼ normal

filter_age (normal)



▼ nested

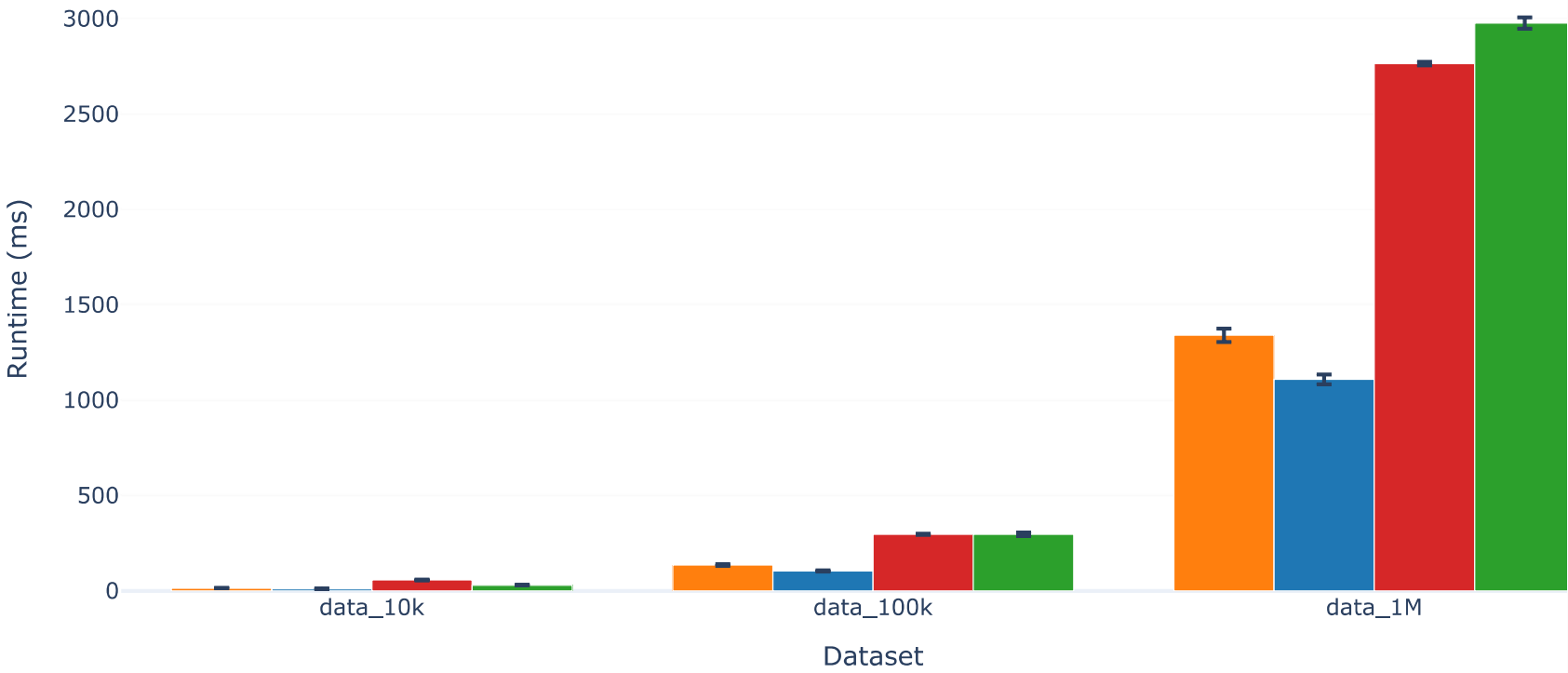
filter_age (nested)



▼ filter_id

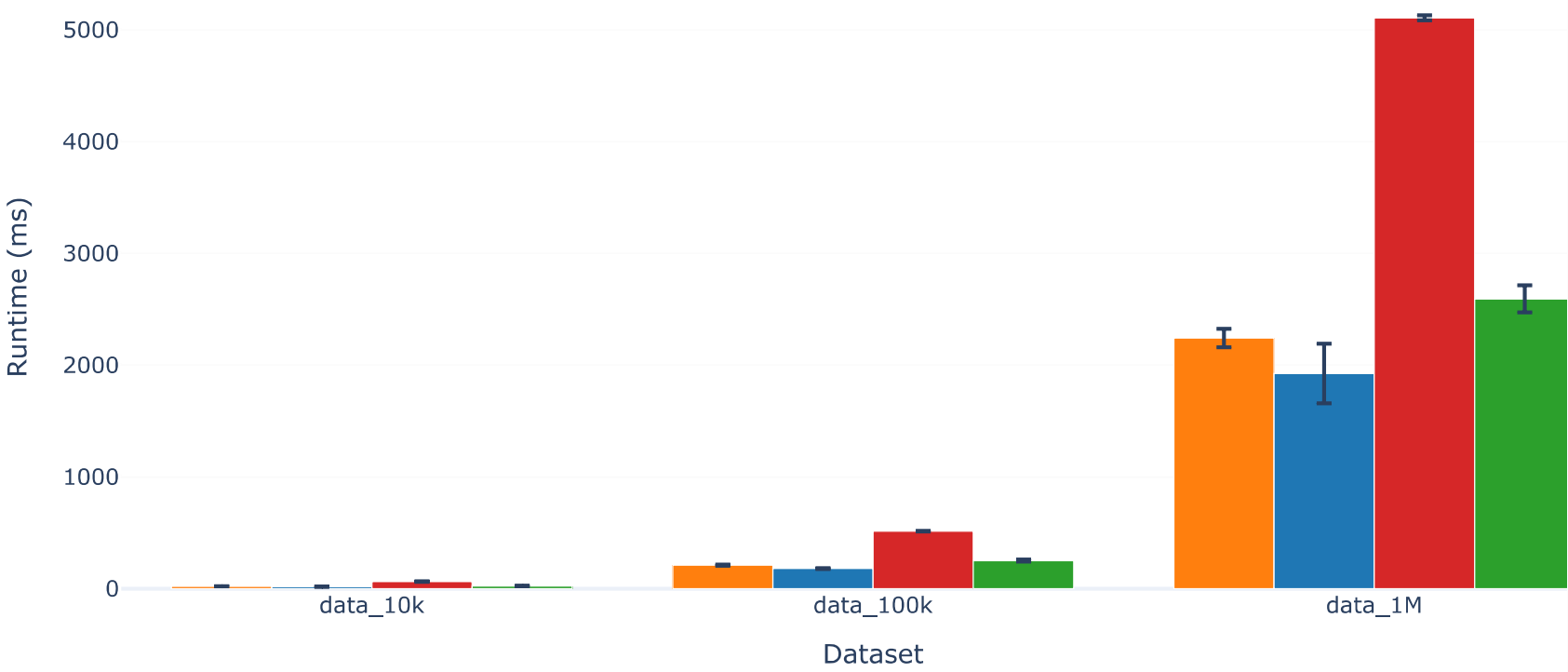
▼ normal

filter_id (normal)



▼ nested

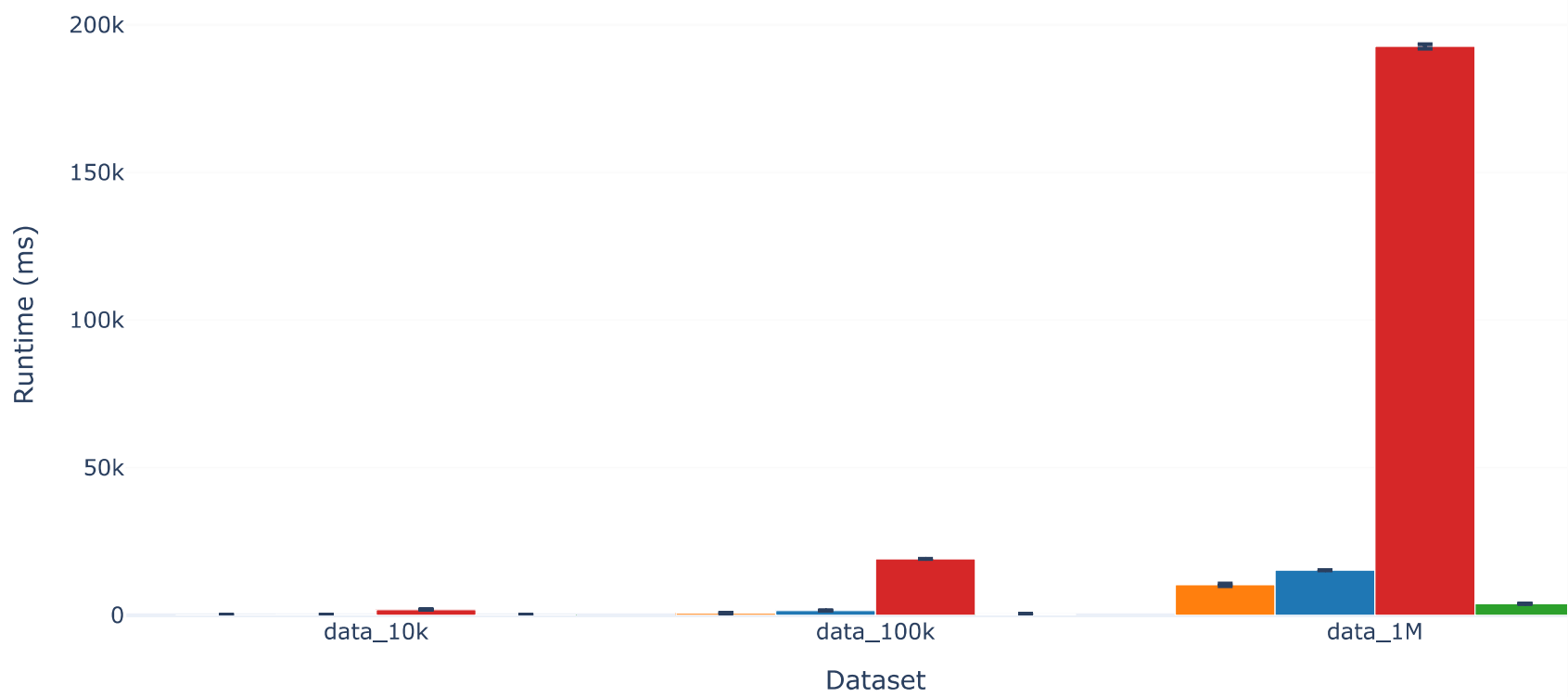
filter_id (nested)



▼ map_div4

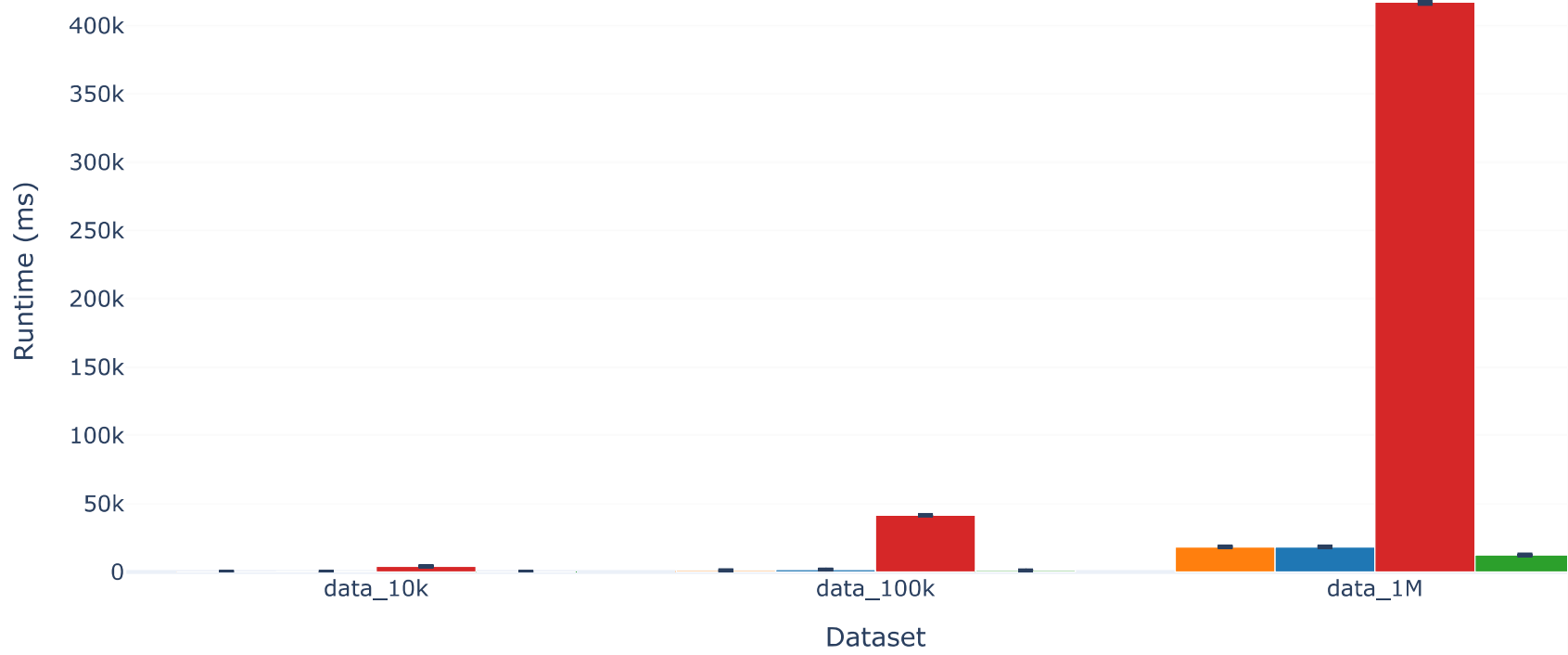
▼ normal

map_div4 (normal)



▼ nested

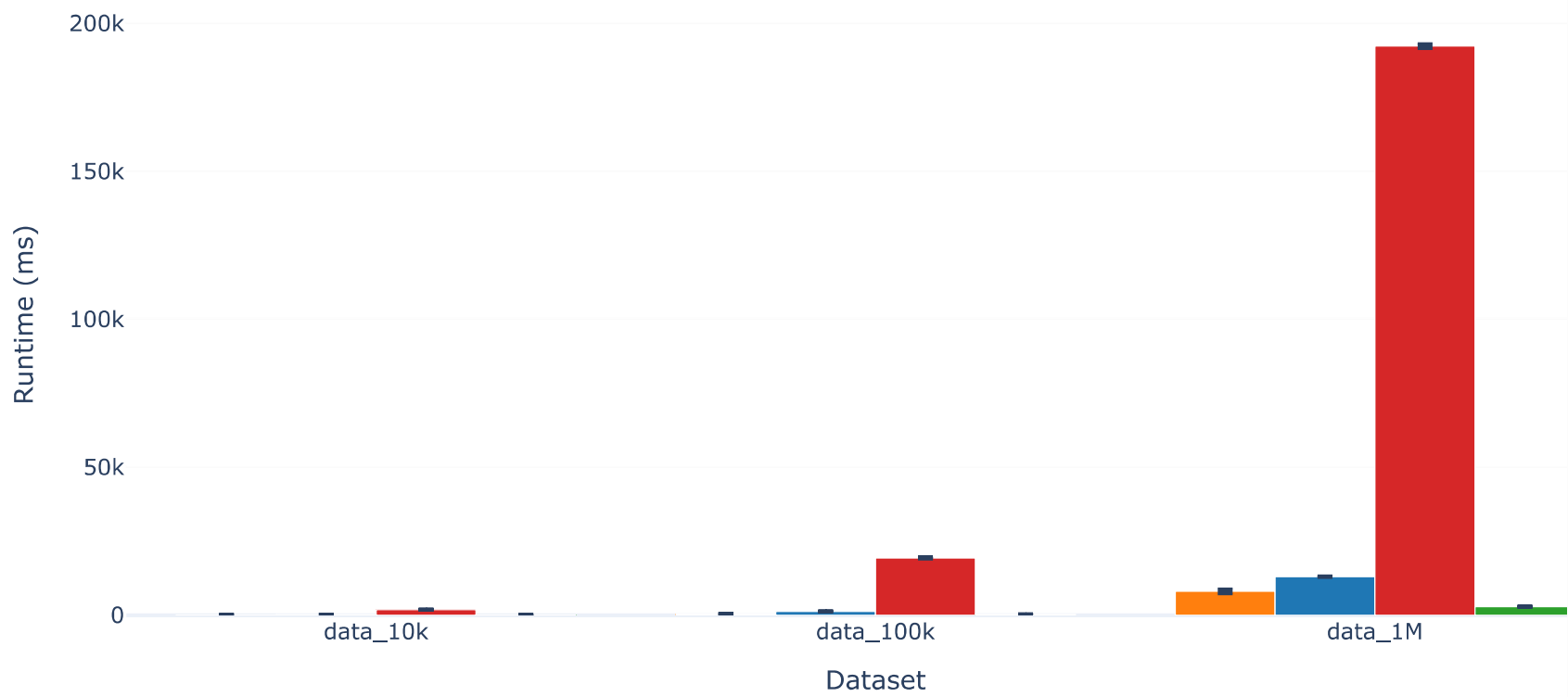
map_div4 (nested)



▼ map_plus100

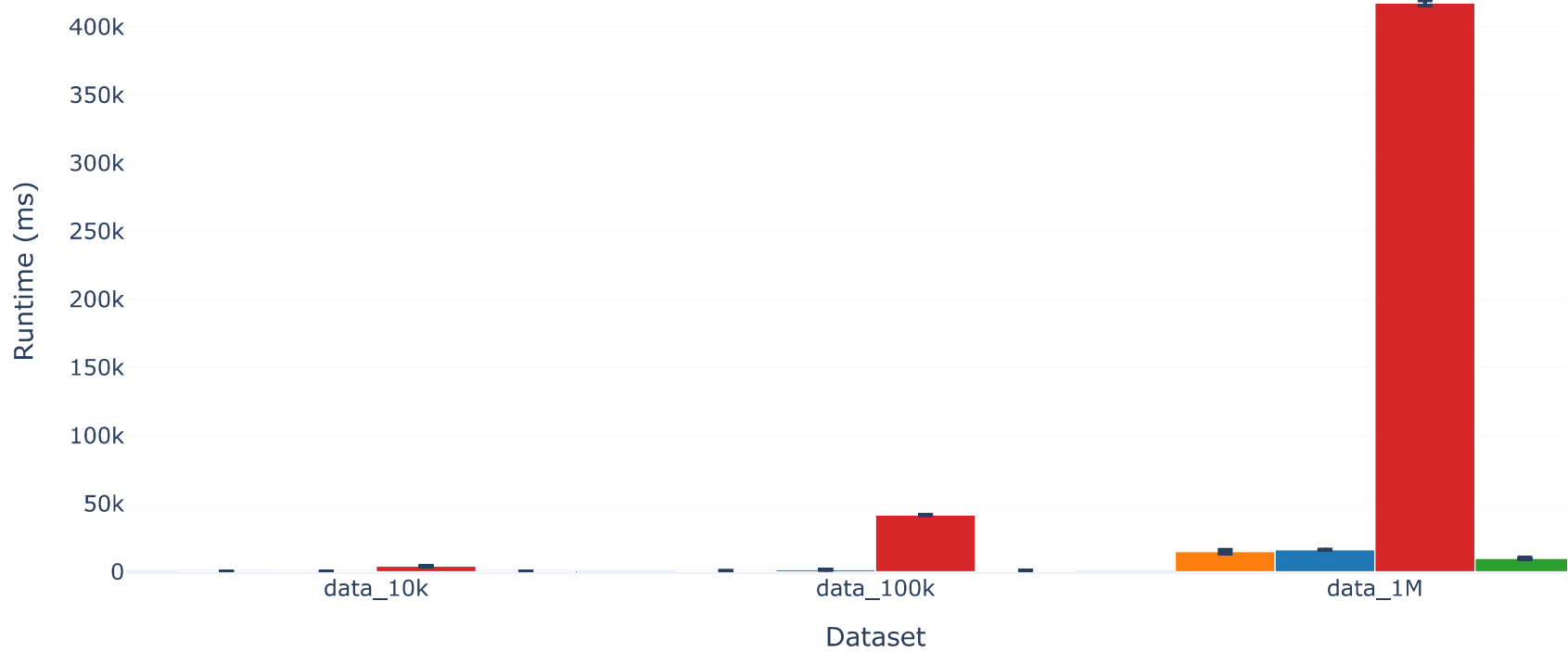
▼ normal

map_plus100 (normal)



▼ nested

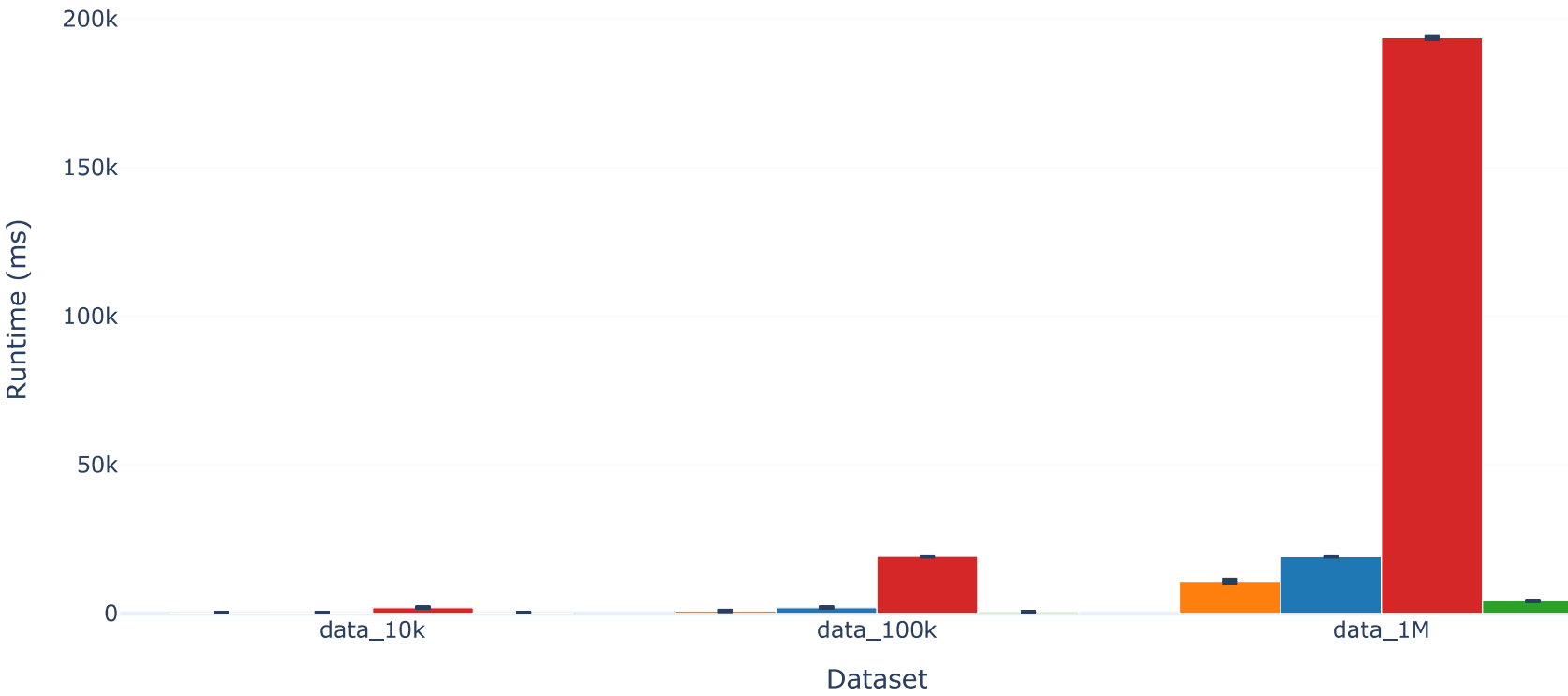
map_plus100 (nested)



▼ map_pow6

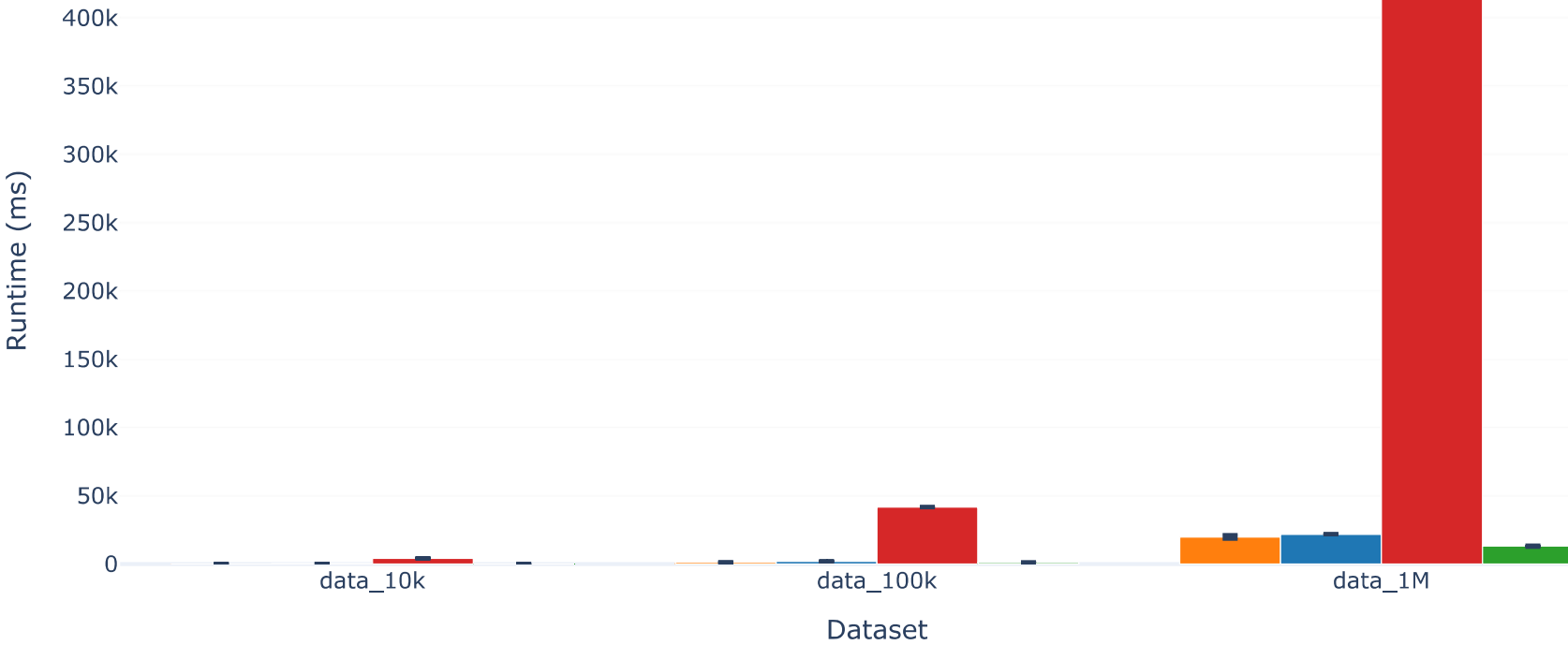
▼ normal

map_pow6 (normal)



▼ nested

map_pow6 (nested)



Conclusions

Since the results are not representative, it would be unfair to draw any concrete conclusions, however we can still compare how each implementation's runtime changed based on the dataset and the operations, which it had to do.

Looking at the `filter` operations, surprisingly the single-threaded implementation proved to be the faster one.

The non-nested `map` operations show that the multi-threaded Java implementations are noticeably faster. With nested structures this difference reduces, but single-threaded is still consistently slower.

On the large (10M record) datasets we can see an expected speed difference between the single- and multi-threaded implementations in the non-nested `filter_id`, and `map` operations. Nested structures were much closer in runtime.

Overall the 3 level nested structure seems to slow down the runtime by roughly 2 times, though this is not consistent throughout the measurements.